The Agent Stack:
Scripts, Programs, Prompts, Skills, Plugins, Agents

Omar

September 2026

1 The model I started with

```
1. Script      (deterministic)
2. Program     (application, bigger than script)
3. Skill       (contains scripts and instructions)
4. Plugin      (contains scripts, instructions, and other things)
5. Agent       (the thing that controls all these)
```

Two things in it are right: deterministic code sits below instruction packages, and the agent is the thing in control. One thing is wrong: this is a single stack. It is not. The *artifacts* stack; the agent does not belong on the stack, it stands *beside* it. And containment runs in every direction, so the whole system is a graph, not a tower.

2 The fix: two axes and one runtime

Every item is one of three kinds:

1. **Deterministic code** (script, program): given input X, produces output Y. No judgment. If it works once, it works forever.
2. **Instruction packages** (prompt, skill, plugin): text plus optional resources that tell a model what to do. Inert until an agent reads them.
3. **Runtime** (agent): a model in a loop, with tools. The only participant that decides anything.

3 Corrected order, low to high

Why the prompt sits between program and skill:

- A script or program needs **no model**: that is why they are below the line.
- A prompt **needs a model but no packaging**: one-shot text.
- A skill needs a model **and** comes with packaging: trigger, resources, reusability.

#	Thing	Kind	Notes
1	Script	code	interpreted glue, no build step
2	Program	code	compiled binary; a real software distinction, but invisible to an agent: both levels 1–2 are just "callable deterministic code"
3	Prompt	instruction (raw)	**new level, inserted here.** Needs a model to interpret, unlike code. No packaging, no trigger, one-shot: dies with the conversation
4	Skill	instruction (packaged)	directory: SKILL.md plus optional scripts/, references/, assets/. A prompt plus resources plus a trigger; reusable; progressively disclosed
5	Plugin	instruction (bundled)	directory: skills/, commands/, agents/, hooks/, MCP config. A distribution format: several skills plus harness extensions
–	Agent	runtime	agent .md definition plus a running process. Not above the others as a container: beside them as the reader and executor

Table 1: Physical forms: script/program are files or binaries; prompt is text; skill and plugin are directories; an agent is a definition file while it is waiting, and a process while it runs.

4 Containment: the stack is really a graph

The second realization was correct: anything can contain anything.

- **program contains agent**: any program that calls an LLM API inside a loop. Claude Code itself is a level-2 program (a compiled CLI) hosting a level-5 agent. The stack loops on itself.
- **skill contains script contains agent**: a skill's script may itself call an LLM API; now a skill contains an agent.
- **skill points to skill**: a skill body can tell the agent to invoke another skill.
- **plugin contains skills, agents, commands, hooks**.
- **CLAUDE.md** is a standing prompt; a **slash command** is a saved prompt with a manual trigger.

So each level is a containment *option*, not a law. From any single node the view is a tree; the whole system is a directed graph. What puts the agent "on top" is not containment but **control**: the agent is the only thing in the picture that decides.

5 Do plugins contain agents?

Yes. A plugin can carry an `agents/` directory:

```
my-plugin/
|-- agents/
|   `-- reviewer.md      <- subagent definition
|-- skills/
```

```
|-- commands/
`-- hooks/
```

`reviewer.md` is a markdown file with YAML frontmatter (name, description, tools, model) and a body of instructions. The harness registers it; the main agent can spawn it as a subagent: a fresh model instance with a fresh context, running that definition. The recursion is real too: a plugin loaded by the main agent's harness can define subagents that the main agent then spawns. The plugin stays inert the whole time; the subagent is a live process.

6 Do agents exist physically, like SKILL.md?

In two senses:

- **Definition**: yes. Literally a `.md` file with frontmatter (`~/.claude/agents/foo.md`, or a plugin's `agents/foo.md`). As inert as SKILL.md.
- **Instance**: no. A running process: a model holding that definition as its instructions, plus tools, plus a task. Exists only while the loop runs. You can `cat` the definition; you can never `cat` the instance.

The symmetry:

```
SKILL.md  : instructions FOR an existing agent
agent.md  : instructions TO HATCH a new agent
```

Both inert until the harness uses them. The harness treats them differently: one is loaded into an existing context, the other spawns a new one.

7 Where the prompt fits in the flow

```
You type a prompt (level 3)
  -> harness assembles context: system prompt + CLAUDE.md + skill
     descriptions
  -> agent reads it, decides
  -> agent runs a script or program (levels 1-2)
  -> or loads a skill body (level 4), possibly from an installed
     plugin (level 5)
  -> or spawns a subagent (another agent .md)
  -> observes the result, repeats
```

The prompt is the **input event** that starts every cycle. Everything else on the instruction axis is a way of storing prompts so they survive: CLAUDE.md (always-on prompt), slash command (manual prompt), skill (self-triggering prompt plus resources), agent definition (a prompt that becomes a whole new mind).

8 Summary

- **Code** (script, program): deterministic, decides nothing.
- **Instructions** (prompt, skill, plugin): text plus resources, decides nothing: packaging at increasing levels of structure.
- **Agent**: the only decider: a model in a loop.

- **Containment** is arbitrary in every direction; **control** is not: everything inert is waiting for an agent to read or run it.